LAB 9 (SECJ1023)

PROGRAMMING TECHNIQUE II

SECTION 03, 04 & 06, SEM 2, 2025/2026

1. Find all errors in each of the following fragments of code.

   a.
```
catch
{
    quotient = divide(num1, num2);
    cout << "The quotient is " << quotient << endl;
}
try (string exceptionString)
{
    cout << exceptionString;
}
```

   b.
```
try
{
    quotient = divide(num1, num2);
}
cout << "The quotient is " << quotient << endl;
catch (string exceptionString)
{
    cout << exceptionString;
}
```

   c.
```
template <class T>
T square(T number)
{
    return T * T;
}
```

   d.
```
template <class T>
int square(int number)
{
    return number * number;
}
```

   e.
```
template <class T1, class T2>
T1 sum(T1 x, T1 y)
{
    return x + y;
}
```

2. Write a function that searches a numeric array for a specified value. The function should return the subscript of the element containing the value if it is found in the array. If the value is not found, the function should throw an exception.

3. Make the function written in Question 2 as template.

4. The following function accepts an integer argument and returns half of its value as a double.

```
double half(int number)
{
    return number / 2.0;
```

```
}
```

Write a template that will implement this function to accept an argument of any type.

5. Given a class template named **`List`**, which is defined as follows:

```
template<class T>
class List
{
    //Members are declared here
};
```

Write an example of how you would use integer value as the data type in the definition of a **`List`** object. (Assume the class has a default constructor.)

6. As the following **`Rectangle`** class is written, the **`width`** and **`length`** members are doubles. Rewrite the class as a template that will accept any data type for these members.

```
class Rectangle
{
    private:
        double width;
        double length;
    public:
        void setData(double w, double l)
        { width = w; length = l;}
        double getWidth()
        { return width; }
        double getLength()
        { return length; }
        double getArea()
        { return width * length; }
};
```

7. Given the following Program 9.1 that handle a special case without exception handling. Rewrite this program to handle exceptions using exception handling.

```
1  //Program 9.1
2  #include <iostream>
3  using namespace std;
4
5  int main( )
6  {
7      int donuts, milk;
8      double dpg;
9      cout << "Enter number of donuts:\n";
10     cin >> donuts;
11     cout << "Enter number of glasses of milk:\n";
12     cin >> milk;
13     if (milk <= 0)
14     {
15         cout << donuts << " donuts, and No Milk!\n"
16              << "Go buy some milk.\n";
17     }
18     else
19     {
20         dpg = donuts/static_cast<double>(milk);
21         cout << donuts << " donuts.\n"
22              << milk << " glasses of milk.\n"
```

```
23                 << "You have " << dpg
24                 << " donuts for each glass of milk.\n";
25       }
26       cout << "End of program.\n";
27       return 0;
28    }
```

8. Program 9.2 is intended to perform array manipulations consisting of adding and removing elements to/from an array. A class named **`Array`** has been defined in the program for this purpose. The array is specified such that it can only hold up to three elements. The elements are stored in the attribute **`data`**, whereas the number of elements currently held by the array is kept in the attribute **`count`**. When an element is added, is increased by 1, and it is decreased by 1 if an element is removed. The class needs to provide a bound-error checking whereby elements cannot be added anymore when the array is already full. Similarly, removing an element cannot be accomplished if the array is empty. Also, accessing an element is allowable only if a valid index is used.

   Complete the program according to the tasks 1 to 9.

```
1   //Program 9.2
2   #include <iostream>
3   #include <exception>
4   using namespace std;
5
6   const int MAX = 3;
7
8   class Array
9   {
10      private:
11         int data[MAX]; //array elements
12         int count; //the number of element currently
13                    //held by the array
14
15      public:
16      //Task 1: Define three exception classes named 'Full',
17      //'Empty' and 'NegativeIndex'
18
19      Array() {count = 0;}
20      int getCount() const {return count;}
21
22      //Method add: To add an element to the array
23      void add(int element)
24      {
25         //Task 2: Throw a  'Full' exception if the array
26        //already holds the maximum number of elements.
27
28          data[count] = element;
29          count++;
30     }
31
32      //Method remove: To remove an element from the array.
33      void remove()
34           {
35          //Task 3: Throw an 'Empty' exception if the array is
```

```
36          //empty, i.e., no item held in the array.
37          count--;
38      }
39
40      //Method displayElement: To display an element
41      //based on its index entered from the keyboard.
42      void displayElement()
43      {
44          int index;
45          cout << "Enter the index of the element you want to"
46               << "display => ";
47          cin >> index;
48
49          //Task 4: Throw a 'NegativeIndex' exception if the
50          // user entered a negative value for the index.
51
52          //Task 5: Throw the user input if the input is larger
53          //than the valid index that the array currently has.
54
55          cout << "Index: " << index <<Element: "
56               << data[index] << endl;
57      }
58  }; // End of class Array
59
60  int main()
61  {
62      Array a;
63
64      a.add(11);
65      cout << "Number 11 has been added. Current number of"
66           << " element = " << a.getCount() << endl;
67
68      a.add(22);
69      cout << "Number 22 has been added. Current number of"
70           << " element = " << a.getCount() << endl;
71
72      cout << endl;
73      try {
74        a.displayElement();
75       }
76
77      //Task 6: Handle the case where the user has entered a
78      //negative index.
79
80      //Task 7: Handle the case where the user has entered the
81      //index that is larger than the valid index
82
83      catch (...){}
84      cout << endl;
85      try {
86        a.add(33);
87        cout << "Number 33 has been added. Current number of"
88             << " element = "  << a.getCount() << endl;
89
90        a.add(44);
91        cout << "Number 44 has been added. Current number of"
```

```
92              << " element = "  << a.getCount() << endl;
93      }
94
95      //Task 8: Handle the case where an element wants to be
96      //added but the array is already full.
97
98      catch (...){}
99      cout << endl;
100     try {
101       a.remove();
102       cout << "An element has been removed. Current number"
103            << " of element = "  << a.getCount() << endl;
104
105       a.remove();
106       cout << "An element has been removed. Current number"
107            << " of element = "  << a.getCount() << endl;
108
109       a.remove();
110       cout << "An element has been removed. Current number of
111       element = "  << a.getCount() << endl;
112
113       a.remove();
114       cout << "An element has been removed. Current number"
115            << " of element = "  << a.getCount() << endl;
116     }
117
118     //Task 9: Handle the case where an element wants to be
119     //removed but the array is empty.
120     catch (...){}
121
122     return 0;
123 }  //End of main function
```

9. Write a template-based function that calculates and returns the absolute value of two numeric values typed in. The function should operate with any numeric data types (e.g. float, int, double, char).

10. Write a template-based class that implements a set of items. The class should allow the user to:

    a. Add a new item to the set.

    b. Get the number of items in the set.

    c. Get a pointer to a dynamically created array containing each item in the set. The caller of this function is responsible for de-allocating the memory.

    Test the class by creating sets of different data types (e.g. integers, strings, etc.)

11. Use std::array to store and process a fixed number of student scores.

    a. Declare an array of 5 double values using std::array.
    b. Prompt the user to enter 5 scores.
    c. Calculate and display:
    - All entered scores
    - The average score
    - The highest score

d. Sample output:

```
Enter 5 scores:
Score 1: 78.5
Score 2: 90
Score 3: 66.5
Score 4: 88
Score 5: 75
Scores entered: 78.5, 90, 66.5, 88, 75
Average score: 79.2
Highest score: 90
```

12. Write a template-based class that implements a set of items. The class should allow the user to: